

SymPy Bug Report

1 Bug 27: solveset returns a false acosh solution outside the principal range

1.1 Minimal Reproducer

```
from sympy import Eq, I, N, S, acosh, pi, solveset, symbols

x = symbols("x")
eq = Eq(acosh(x), 2*pi*I)
sol = solveset(eq, x, domain=S.Complexes)
print("solution =", sol)
for s in sol:
    print("residual at", s, "=", N(acosh(s) - 2*pi*I, 80))
```

1.2 Observed Behavior

```
solution = {1}
residual at 1 =
-6.283185307179586476925286766559005768394338798750211641949889184615632812572418*I
```

SymPy returns the finite solution set $\{1\}$. Direct substitution shows that this returned point does not satisfy the original equation: the residual is $-2\pi i$, displayed above numerically to 80 digits.

1.3 Expected Behavior

The correct result is $\emptyset$. The principal value satisfies $\operatorname{acosh}(1) = 0$, not $2\pi i$, and there is no complex number x whose principal inverse hyperbolic cosine is $2\pi i$.

1.4 Mathematical Explanation

The error comes from treating $\cosh$ as a global inverse of the principal function acosh . Although

$$\cosh(2\pi i) = 1,$$

this only proves that 1 solves the composed equation

$$\cosh(\operatorname{acosh}(x)) = \cosh(2\pi i).$$

It does not prove that 1 solves the original principal-branch equation

$$\operatorname{acosh}(x) = 2\pi i.$$

For the returned candidate,

$$\operatorname{acosh}(1) - 2\pi i = -2\pi i \neq 0.$$

Thus the result is not an alternative branch representation or a harmless simplification; it is a false solution introduced by discarding the range condition of the principal inverse hyperbolic cosine.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant solver logic in `sympy/solvers/solveset.py`. The key function is `_invert_complex` around lines 550–557, with the call reaching `_solveset` around line 1313. The traced call path rewrites the input as $\operatorname{acosh}(x) - 2\pi i = 0$, removes the additive $-2\pi i$, and then inverts $\operatorname{acosh}(x) = 2\pi i$ through the generic complex inversion path. That path asks $\operatorname{acosh}(x)$ for its inverse. The inverse method for acosh , located in the file `sympy/functions/elementary/hyperbolic.py`, returns `cosh`.

The problematic rule is the generic mapping of the target set through the function returned by `inverse()`:

```
if hasattr(f, 'inverse') and f.inverse() is not None:
    return x, imageset(Lambda(n, f.inverse()(n)), g_ys)
```

For this equation, that rule maps the target set $\{2\pi i\}$ through `cosh`, producing $\{\cosh(2\pi i)\} = \{1\}$. The diagnosis notes that this treats `cosh` as though it were a globally valid inverse for principal `acosh`. Since `acosh` has a principal range, the inversion is only sound when the right-hand side is in that range. Here it is not, so the generic inversion solves the composed periodic equation instead of the original equation.

A plausible fix direction supported by the diagnosis is to exclude inverse hyperbolic functions from this branch-blind generic inverse rule, or to add range conditions for the target values before mapping through the elementary inverse. The diagnosis also notes that a residual check on finite candidates would prevent this particular false solution from being returned.

1.6 Additional Failing Instances

The independent verification covers the representative target $2\pi i$ and five additional targets of the same form. In each case SymPy maps the target through `cosh`, returning 1 or -1 , but the principal value of `acosh` at the returned candidate is 0 or πi , not the requested target. The expected behavior is therefore the same across the family: the listed false candidate must not be reported as a solution.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset(acosh(x) = 2πi)</code>	returns a set containing 1	1 is not a solution; $\operatorname{acosh}(1) = 0$
<code>solveset(acosh(x) = -2πi)</code>	returns a set containing 1	1 is not a solution; $\operatorname{acosh}(1) = 0$
<code>solveset(acosh(x) = 3πi)</code>	returns a set containing -1	-1 is not a solution; $\operatorname{acosh}(-1) = \pi i$
<code>solveset(acosh(x) = -3πi)</code>	returns a set containing -1	-1 is not a solution; $\operatorname{acosh}(-1) = \pi i$
<code>solveset(acosh(x) = 4πi)</code>	returns a set containing 1	1 is not a solution; $\operatorname{acosh}(1) = 0$
<code>solveset(acosh(x) = -4πi)</code>	returns a set containing 1	1 is not a solution; $\operatorname{acosh}(1) = 0$

1.7 Related Issues Assessment

The best-effort deduplication check found public material on the relevant principal-branch mathematics for inverse trigonometric and inverse hyperbolic functions, but did not identify a public SymPy issue or fix for the exact $\operatorname{acosh}(x) = 2\pi i$ false-solution reproducer. The verdict is therefore a new instance of a known failure mode: the broader range-condition issue is recognizable, but this concrete reproducible behavior is previously unreported in the available evidence and remains individually actionable as a SymPy issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Eq, I, S, acosh, pi, simplify, solveset, symbols

x = symbols("x")

CASES = [
    (2*pi*I, 1),
    (-2*pi*I, 1),
    (3*pi*I, -1),
    (-3*pi*I, -1),
    (4*pi*I, 1),
    (-4*pi*I, 1),
]

@pytest.mark.parametrize("target,bad", CASES)
def test_solveset_acosh_rejects_values_outside_principal_range(target, bad):
    sol = solveset(Eq(acosh(x), target), x, domain=S.Complexes)

    assert sol.contains(bad) is not S.true
    assert simplify(acosh(bad) - target) != 0
```